21. Connection Pooling

For web sites with high traffic, resources to acquire a database connection can be expensive. Also, imagine a situation where there are 500 concurrent users of a web site and each is trying to get hold of a connection to select or update data. In this scenario, to prevent multiple users to corrupt the data, the database has locks. When a user does a select on a table, the table is locked. No other user can get hold of the table unless the lock is freed. For 500 users to wait for a connection is not acceptable.

Connection pooling in Java is a technique used to improve the performance of database-driven applications. Instead of creating a new connection every time the application needs to interact with the database, a pool of connections is created and managed by a connection pool manager. This eliminates the overhead of establishing a new connection each time, resulting in faster response times and better resource utilization.

21.1 Advantages of connection pooling

Following are the advantages of the connection pooling.

- **Improved performance** − Connection pooling reduces the overhead of creating and closing connections, resulting in faster response times for your application.
- **Resource optimization** − By reusing connections, connection pooling helps to conserve system resources, such as memory and network connections.
- **Scalability** − Connection pools can be configured to grow or shrink dynamically based on demand, allowing your application to handle varying workloads.

21.2 Popular Connection Pooling Libraries

Following are popular connection pooling libraries.

- **HikariCP** − Known for its performance and efficiency.
- **Apache Commons DBCP** − Widely used and well-established.
- **c3p0** − Another popular option with a focus on stability.

Example for Connection Pooling Using HikariCP

We will use HikariCP connection pool library. In Eclipse, go to File -> New -> Java Project.

Set the Name of project as HikariCPExample. Click Finish. Now in Project explorer, right click on the project name, select Build Path -> Add External Archives. Add following 3 jars:

- HikariCP-5.0.1.jar − Download from HikariCP-5.0.1.jar
- mysql-connector-j-8.4.0.jar − Download from mysql-connector-j-8.4.0.jar
- slf4j-api-2.1.0-alpha1.jar − Download from slf4j-api-2.1.0-alpha1.jar

We've created a HikariConfig() instance and set JDBC url, username and password. Maximum pool size is set as 10. getPooledConnection() method is used to get a database connection.

```java
package com.lokesh;

import java.sql.Connection;
import java.sql.SQLException;
import com.zaxxer.hikari.HikariConfig;
import com.zaxxer.hikari.HikariDataSource;

public class HikariCPManager {
	public static HikariDataSource dataSource;
	public static HikariConfig config = null;

	// Set properties in constructor
	HikariCPManager() {
		config = new HikariConfig();
		config.setJdbcUrl("jdbc:mysql://localhost/organization");
		config.setUsername("root");
		config.setPassword("root");
		// Set maximum connection pool size
		config.setMaximumPoolSize(10);
		dataSource = new HikariDataSource(config);
	}

	public static Connection getPooledConnection() throws SQLException {
		return dataSource.getConnection();
	}

	public static void close() {
		if (dataSource != null) {
			dataSource.close();
		}
	}
}
```

In this example, we've created HikariCPManager instance and using getPooledConnection(), we've prepared a pooled connection. Once connection object is ready, we've prepared a PreparedStatement instance with a SELECT query. Using executeQuery(), we've executed the query and printed all the employees by iterating the resultSet received.

```java
package com.lokesh;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class TestPooledConnection {
	public static void main(String[] args) {
		Connection conn = null;
		PreparedStatement pstmt = null;
		ResultSet rs = null;
		try {
			HikariCPManager hcpm = new HikariCPManager();
			conn = hcpm.getPooledConnection();

			if (conn != null) {
				// Prepare statement
				String sql = "SELECT * FROM employees";
				pstmt = conn.prepareStatement(sql);

				// Execute query
				rs = pstmt.executeQuery();

				// Process and print results
```

```
                        while (rs.next()) {
                                int id = rs.getInt("id");
                                String fname = rs.getString("first");
                                String lname = rs.getString("last");
                                System.out.println("ID: " + id + ", First Name: " +
                                                fname + ",  Last Name:  " + lname);
                        }
                } else {
                        System.out.println("Error getting connection.");
                }
        } catch (SQLException e) {
                e.printStackTrace();
        } finally {
                // Close all resources
                try {
                        if (rs != null)
                                rs.close();
                        if (pstmt != null)
                                pstmt.close();
                        if (conn != null)
                                conn.close();
                } catch (SQLException e) {
                        e.printStackTrace();
                }
        }
        // Close connection pool
        HikariCPManager.close();
    }
}
```

Output:

```
SLF4J(I): Connected with provider of type [org.apache.logging.slf4j.SLF4JServiceProvider]
ID: 1, First Name: Rita,  Last Name:  Tez
ID: 2, First Name: Sita,  Last Name:  Singh
ID: 3, First Name: Zaid,  Last Name:  Khan
ID: 4, First Name: Sumit,  Last Name:  Mittal
ID: 5, First Name: John,  Last Name:  Paul
ID: 7, First Name: Sita,  Last Name:  Singh
```